

Research

AI chatbots in programming education: guiding success or encouraging plagiarism

Gökhan Akçapınar¹  · Elif Sidan¹ 

Received: 4 September 2024 / Accepted: 12 November 2024

Published online: 21 November 2024

© The Author(s) 2024 

Abstract

This study examines the impact of an AI programming assistant on students' exam scores and their tendency to accept incorrect AI-generated information. The customized AI programming assistant was developed by the authors using GPT based Large Language Model (LLM). A one group pretest–posttest quasi-experimental design was utilized to answer research questions. Students were asked to take identical programming exams twice: once without AI assistance and once with the option to use the AI assistant. Results showed that the students' average exam scores significantly increased from 48.33 to 74.47 with a large effect size ($d = 1.56$) when they used the AI assistance. On the other hand, when student–AI interaction logs were analyzed for a specific question, it was found that AI generated incorrect answers to 36 students. Thirty-three of these students (92%) answered the question incorrectly. Even more interestingly, despite the AI-generated response containing an obvious error, 22 of them (61%) copied and pasted the AI's response directly into the answer field. Only 3 students (8%) ignored the incorrect response generated by the AI and answered the question correctly. A significant portion of students accepting incorrect information provided by AI underscores the need for careful integration of AI tools into learning environments. Moreover, our findings emphasize the importance of specially developed AI tools rather than free tools like ChatGPT in exploring the new type of interaction between students and AI.

Keywords Large language models · AI in education · GPT · AI programming assistant · Plagiarism · Student-AI interaction

1 Introduction

Despite improvements in the methods and tools used in programming education, the dropout and failure rates of learners are still high [1–3]. Studies indicate that students face various problems in introductory programming courses resulting in dropouts and failure. Inadequate feedback [4] and a lack of fundamental programming skills [5] are among the main reasons for the failure. On the other hand, emerging developments of technology bring new insights with brand new applications not only applied sciences but also educational practices [6]. The transformation in the technologies used creates a new perspective on the difficulties experienced in education and offers different solutions. The main next generation technologies that can be used in education will be Generative Artificial Intelligence (GenAI) technologies such as Large Language Models (LLMs). In a varied range of use, programming education is one of the areas where LLMs are powerful. Because in the training process of LLMs like GPT (Generative Pre-trained Transformers), which drives ChatGPT, code from open-source projects stored in repositories like GitHub is used as well as text written in various

✉ Gökhan Akçapınar, gokhana@hacettepe.edu.tr; Elif Sidan, elifsidan@hacettepe.edu.tr | ¹Department of Computer Education and Instructional Technology, Hacettepe University, 06800 Ankara, Türkiye.



natural languages. When the challenges are taken into consideration while the students are learning programming, the use of AI assistants based on LLMs hosts a bright future in programming education. However, it is not clearly known today how these assistants will be adapted to education and how their effects can be improved [7]. Moreover, there are several challenges in integrating generative AI models into educational settings. One of the most significant problems for text-to-text models is hallucination [8]. In other words, these models can generate false or inaccurate information, which students need to approach critically. Otherwise, this issue can lead to students learning incorrect information. Another significant problem is plagiarism. The human-like text generation capability of large language models makes it difficult for plagiarism detection software to identify generated content. This could lead to an increase in academically dishonest behaviors among students. Plagiarism is also one of the most cited problems in programming education and AI tools have a potential to increase plagiarism [9]. GenAI tools have a potential to be used as a plagiarism tool to hide the lack of knowledge [10]. Students tend to have higher grades in a less task completion time when they plagiarized with the help of AI in an assessment [11].

In summary, the success of LLMs in programming tasks like coding and debugging does not mean that students improve their learning by using these models. Although studies have shown that AI-based tools can enhance student performance, these studies are limited to students' test scores or self-reported perceptions. It is also noteworthy that in most of these studies, general-purpose AI tools like ChatGPT are used [12] instead of AI tools specifically designed for programming tasks. Since the development and use of these tools are not under the control of instructors, an in-depth analysis of student-AI interactions is not possible. Based on this, the planned study analyzed student interactions with an AI Programming Assistant developed by authors in a controlled learning environment, seeking answers to the research questions presented below.

1. Does the use of an AI Assistant significantly improve students' exam scores compared to their performance without AI assistance?
2. How likely are students to accept and incorporate incorrect information provided by the AI Assistant in their responses?

2 Literature review

This section includes studies that examine the positive and negative impacts of generative AI-based assistants in programming education. For example, in an experimental study conducted by Kazemitabaar et al. [13], it was aimed to measure the coding performance of using an AI programming assistant on students who were new to programming and had no previous programming knowledge. In the research conducted with a total of 69 students between the ages of 10–17 using the OpenAI Codex for teaching Python programming language, students completed 45 different programming tasks on basic concepts, conditionals, loops and arrays, consisting of steps such as real-time code editing, error finding and debugging, and the differences between the codes created by the group using Codex and the group not using Codex were examined. When the code editing performance of the students who used Codex Assistant was higher than the students who did not use the assistant, it is suggested that AI assistants can be used to improve student performance in programming education. In the research conducted by Okonkwo and Ade-Ibijola [14], a tool was created to provide learning support in Python programming language education. The learning support provided by this tool called "PythonBot" to the students in Python programming language education was investigated, and as a result, it was determined that it had a facilitating effect on the majority of the learners and improved their programming skills. In their research, Pankiewicz and Baker [15] used a feedback system using the GPT-3.5 model in their object-oriented programming course in C# programming language and integrated this system into the course content to be used in the assignment part of the course. They investigated whether the system improved learner performance and learning experience by providing learners with feedback that provided hints and additional information to help them in the process of completing their assignments and found that the experimental group tended to complete the assignments in less time and that the system used positively affected the learning experience and learning performance. In their research, Eilermann et al. [16] introduced an AI assistant called 'KIAAA', which was developed to be used in the programming course focused on automation engineering education as an artificial intelligence-based intelligent teaching system. This designed assistant has been developed to analyze student profiles within the scope of programming education, to create individual tasks according to student knowledge level and preferences and to present them to students. The research supports that the proposed intelligent teaching systems will improve students' programming skills in interdisciplinary

departments such as automation engineering and will contribute to increasing motivation and interest in the course by providing instant feedback to students. Kazemitabaar et al. [17], developed a programming assistant named Code-Aid for the use of Generative Artificial Intelligence in programming education. With the implementation of this tool, it was aimed to determine the frequency of students' use of the assistant, which features of the tool they prefer, their usage patterns and the qualities of the questions they ask, how much technical support the tool provides to students without directly showing the codes and its effectiveness, students' experiences compared to other tools, how teachers will integrate the assistant into the course, and to evaluate student and instructor opinions about the tool. The research suggests that AI-based assistants can be used in programming education to provide educational support to students and increase the applicability of the education. Cipriano and Alves [18], examined learners' code writing assignments using the GPT-3 model in an object-oriented programming course and reported that the model can be used to validate assignments, adjust the difficulty of assignments and create automated tests for assignments, and recommended that the GPT-3 model should be incorporated into the classroom to help learners critically evaluate the solutions provided by the model and thus lead to higher quality code production.

In addition to the positive effects of GenAI-based assistants in programming education, studies highlighting the potential negative consequences they may create have also gained prominence. For example, in an experimental study, Chen et al. [19] suggest that after the emergence of GenAI tools such as ChatGPT, students' tendency to plagiarism has shifted from online environments to AI tools, resulting in losses in deep and meaningful learning. Denzler et al. [20] searched about the code plagiarism rates in a CS1 course and by detecting some code structures which hadn't been taught to students before, they reached an important finding that use of GenAI tools increases the plagiarism in a meaningful rate. Taylor et al. [21] conducted an experimental study on the plagiarism of the codes created by ChatGPT within an entry-level computer programming course and found that ChatGPT codes can be meaningfully compiled by the codes created by students, making the AI-generated codes hard to be detected.

As can be seen from the studies conducted, today's GenAI tools possess a certain capability in programming and have a high potential to support students' programming education from various perspectives. However, more research is needed to ensure that students benefit from these tools to the maximum extent. Therefore, the purpose of this presented study is to provide data-driven insights to contribute to the current state of the art regarding students' usage of AI assistants especially in programming education. For this purpose, an AI assistant has been developed by the authors. The tool being developed will have a key difference from existing tools: it will be able to record the interactions between students and artificial intelligence (AI). In educational research, such data can be used to answer many research questions. This data can also be used to understand student-AI interactions, a new type of interaction, and to design data-driven interventions to improve these interactions. Another advantage of the tool over existing ones is that it will allow customization of the AI assistant using prompt engineering methods. In other words, the assistant can be tailored to support students' learning processes. This ensures that students receive help specifically related to course content, unlike general-purpose AI tools, which are designed to perform various tasks and can potentially distract students with unrelated elements.

3 Method

In this study, we aim to investigate whether the use of an AI Assistant significantly enhances students' exam scores compared to their performance without AI support. Additionally, we will examine the likelihood of students accepting and incorporating incorrect information provided by the AI Assistant in their responses.

3.1 Participants and learning context

The study was conducted with first-year students enrolled in the Educational Technologies department at a public university in Türkiye. Forty-five students (26 male, 19 female) enrolled in the Introductory Programming course participated in the study. The course content involved teaching basic programming concepts using the C# programming language. Initially, fundamental concepts such as instruction, algorithm, and the compiling process were addressed. Subsequently, topics such as Console I/O operations, variables, conditionals, loops, arrays, methods, file operations, and debugging were covered. In the course, we used .NET 8.0, C# 12.0, and the Visual Studio 2022 development environment. Most of the students did not have prior knowledge of programming, and this course was their first programming class at the university level. The ethical approval was given by the Ethics Committee at Hacettepe University with informed consent from all subjects (code:E-51944218-050-00003573478).

3.2 Procedure

The data used in the study were collected in the 12th week of the 14-week course. This week was chosen to ensure that students had reached a certain level of knowledge in programming. Two weeks before the experiment, the AI assistant was introduced to the students, and they were given training on how to use it in their programming course. To allow students to experience different uses of the AI assistant, they were asked to complete some small tasks before the data collection process. The data collection process was structured as two 30-min exam sessions as part of the midterm exam. In both sessions, students were asked to answer a programming test consisting of the same questions. In the first session, students answered the test questions with their existing knowledge, while in the second session, they were informed that they could use the AI assistant for help if they wished. The decision to use the AI assistant in the second session was left to the students' discretion. Students were also informed that their midterm grades would be the average of the scores they received from these two attempts. The exams were conducted in a computer lab, face-to-face, using the Moodle Learning Management System. Two proctors were present in the classroom during the exam. The questions were set to appear in random order.

3.3 AI programming assistant

A specially developed AI assistant created by the authors was used in this study. The components of the assistant and their interactions are schematically shown in Fig. 1. The diagram represents a framework for integrating an AI Assistant into a student-facing system, comprising several key components: students, the AI Assistant, an API, an LLM (Large Language Model) endpoint, and a log database. Students interact with the AI Assistant, which processes their prompts and provides responses. The AI Assistant serves as the interface, handling natural language understanding and generation to communicate effectively with users. The API acts as a bridge between the AI Assistant and the LLM Endpoint, ensuring secure and efficient data transfer, handling requests and responses, rate limiting, and error handling. The LLM Endpoint, powered by OpenAI's GPT-3.5 Turbo, provides sophisticated language processing capabilities, generating human-like text responses to user prompts. Additionally, all interactions are logged in a Microsoft SQL Server database, capturing details such as prompt, response text, timestamps, and user identifiers. This logging supports monitoring student-AI interaction and providing data for this research. During the training sessions, students were informed that their AI interactions would be recorded, and their consent was obtained.

Students can access the assistant through a link added to the Moodle learning environment without any external user login. The interface of the assistant is designed to be as simple and user-friendly as possible, similar to chatbots like ChatGPT that students frequently use. The large language model working in the background is customized to answer questions only about C# programming language to prevent distractions by any other topic. The assistant can be used for Conceptual Explanations, Code Creation, Code Completion, and Code Explanation.

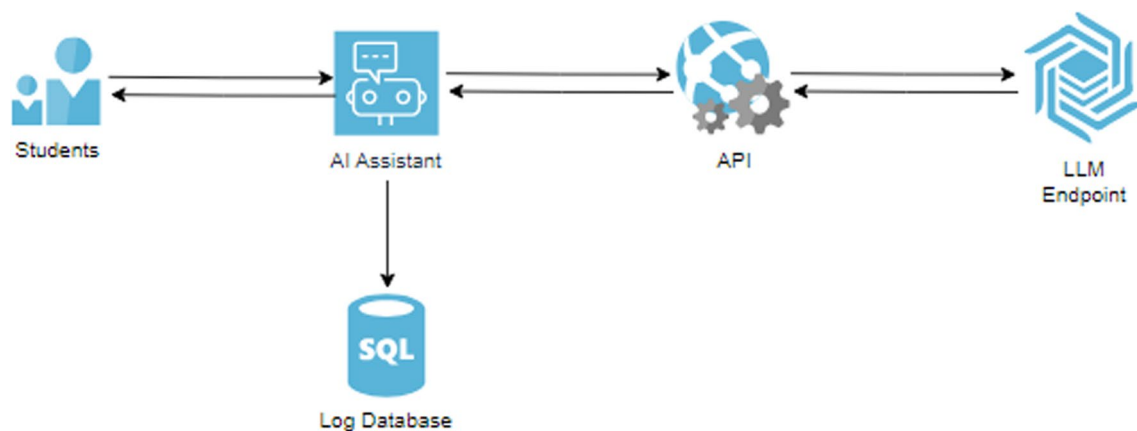


Fig. 1 System design of AI programming assistant

3.4 Programming exam

A programming exam was used to answer research questions. In the programming exam, students were asked 13 questions. Of these, 7 were short-answer questions, and 6 were open-ended questions. The exam covered topics such as mathematical and logical operators, conditionals, type conversions, for and while loops, and arrays. According to Bloom's taxonomy, the questions ranged from the knowledge level to higher-order cognitive skills requiring analysis/synthesis. These included tasks such as mentally compiling a given code to produce its output and rewriting a given code block in a different way (e.g., using a While loop instead of a For loop). Short-answer questions were scored at 5 points each, while open-ended questions were scored at 10 points each.

Within the scope of the first research question, the scores of students on a programming exam when they did not use an AI Assistant and when they did use an AI Assistant were compared. For the second research question, the focus was on how students utilized incorrect content generated by the AI. To investigate this, a question known to be answered incorrectly by the AI was included in the exam. This question is a type conversion question and asks whether converting the String value "111" to an integer using the `Convert.ToByte()` method is a valid type conversion. Although it is a tricky question, it is a valid type conversion in the C# language, but the GPT-3.5 Turbo large language model used in the AI programming assistant answers this incorrectly. The question most students asked, and the answer provided by the AI are shown in Fig. 2.

As seen in the image, the AI generates a response indicating that this type of conversion is not valid. It explains that 111 is outside the limits of the byte data type. Furthermore, the AI generates a clearly erroneous response, suggesting that the 111 is not in the range of 0–255. The second research question analyzed how students used this response.

3.5 Data analysis

In order to answer the first research question, a paired sample t-test was used to compare whether there is a significant difference between students' test scores when they did not use AI and when they did. Cohen's d was calculated

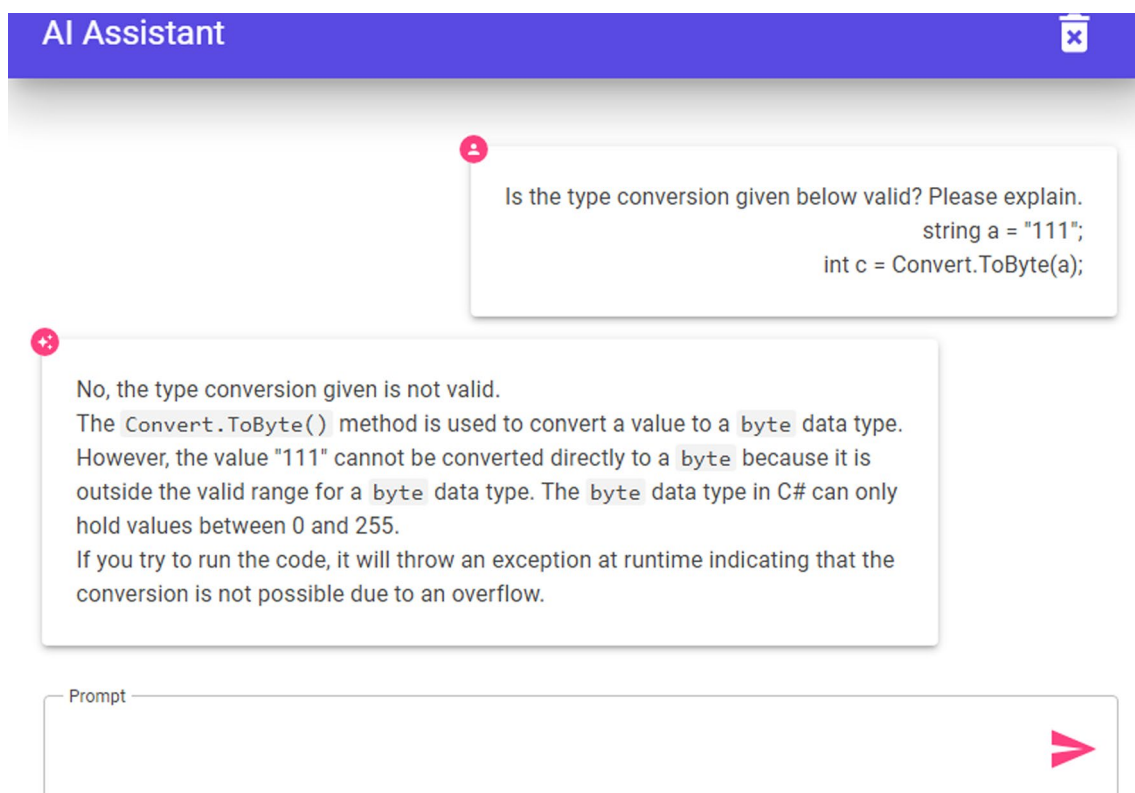


Fig. 2 The question and the AI programming assistant's response containing misinformation

Table 1 Paired sample statistics

	Mean	N	Std. deviation	Std. error mean
First attempt	48.333	45	18.931	2.822
Second attempt	74.466	45	15.028	2.240

Table 2 Paired sample *t*-test results

	Paired differences				<i>t</i>	df	Sig. (2-tailed)
	Mean	Std. deviation	Std. error mean	95% Confidence Inter- val of the Difference			
				Lower Upper			
First attempt–second attempt	–26.133	16.698	2.489	–31.150 –21.116	–10.498	44	0.000

as a measure of effect size [22]. For the second research question, the responses of students to the specified type conversion question when they used AI and when they did not were examined. To do this, students' answers before and after using AI were first coded as correct or incorrect. Based on their answers, students could fall into one of the following four groups: those who answered correctly in both the first and second trials, those who answered correctly in the first trial but incorrectly in the second, those who answered incorrectly in both trials, and those who answered incorrectly in the first trial but correctly in the second. To address the research question, these data were organized into a 2×2 contingency table. McNemar's test was then used to investigate whether there was a significant difference between students' responses in the first and second trials. The McNemar test is known as the chi-square test for the paired samples and can be used to examine paired dichotomous data [23]. Lastly, in the second session, a qualitative analysis was conducted on the students' responses to the selected question and their interactions with the AI assistant to determine whether they directly used the content generated by the AI. The students' answers to the questions were exported from the Moodle learning management system. The questions asked to the AI assistant by the students and the responses from the assistant are recorded along with the student IDs and timestamps. These records were used for the analysis of AI interactions. The data were analyzed by the authors.

4 Results

4.1 AI assistant effect on students' exam scores

Before conducting the data analysis, the assumptions for the paired sample *t*-test were first examined. The dependent variable, test score, was measured on a continuous scale. The independent variable, attempt, consisted of two categorical, related groups (exam scores of the same students in the first and second attempts). To assess normality, a Shapiro–Wilk test was performed. The results indicated no evidence of non-normality for the First Attempt Scores ($W = 0.975$, $p = 0.446$) and Second Attempt Scores ($W = 0.966$, $p = 0.211$). According to the descriptive statistics given in Table 1, in the first attempt, the average score of students is 48.33, with a standard deviation of 18.93 and a standard error mean of 2.82. The number of students who participated in this attempt is 45. In the second attempt, the average score of the students is 74.47, with a standard deviation of 15.03 and a standard error mean of 2.24. The number of students who participated in this attempt is 45.

The results of the paired sample *t*-test are given in Table 2. As is seen in the table, the mean difference between the second and first attempts was 26.13, with a standard deviation of 16.70. The *t*-value was 10.50 with 44 degrees of freedom, and the *p*-value was 0.000, indicating that the difference in scores between the two attempts was statistically significant. The effect size, as measured by Cohen's *d*, was $d = 1.56$, indicating a large effect.

There was a moderate positive correlation ($r = 0.537$, $p < 0.001$) between the scores of the first and second attempts, indicating that students' performances were fairly consistent across both attempts.

4.2 Students' reliance on AI's mistake

Table 3 presents a cross-tabulation of student responses to a specific exam question during two attempts: the first attempt without AI assistance and the second attempt with the option to use AI assistance. During the first attempt, 30 responses (66.7% of total) were incorrect, and 15 responses (33.3%) were correct. During the second attempt, overall, there were 36 incorrect responses (80.0% of total) and 9 correct responses (20.0% of total). Although there is an increase in the incorrect answer during the second attempt, McNemar's test results indicate that there was no statistically significant change in the students' responses after using the AI Assistant. The p-value (Exact Sig. 2-tailed) is 0.146, which is greater than the alpha level of 0.05. This suggests that the proportion of correct and incorrect responses did not differ significantly between the first and second attempts.

We conducted a qualitative analysis of students' interactions during the exam to understand whether they directly used the information generated by the AI or demonstrated critical thinking abilities in identifying and addressing such misinformation. The findings are as follows: All students, except for 3 (93%), used the AI assistant to answer the question. Of the 45 students, 42 used the AI to help answer the question. The AI gave incorrect answers to 36 of these students. Nevertheless, 22 out of the 36 students (61%) copied the AI's response as it was and pasted it as their answer. Eleven students (31%) did not directly copy it but answered the question incorrectly based on the AI's suggestion. Only 3 students (8%) ignored the incorrect response generated by the AI and answered the question correctly. The other 6 students who answered correctly used a different prompt, so the AI did not generate an incorrect answer, allowing them to answer the question correctly.

5 Discussion

In this study, the first focus was on the impact of the AI assistant on students' programming exam scores. As a result, it was found that the average exam scores of students increased from 48.33 to 74.47 when they used AI assistant, which corresponds to a large effect size according to Cohen's d. In other words, there was an average increase of 26% in students' exam scores when they used AI, which is an increase of more than one standard deviation. This is an expected result because AI models are quite successful in solving programming questions. For instance, in the study conducted by Finnie-Ansley et al. [24], the performance of the OpenAI Codex model on programming tasks was examined. The research involved answering actual programming exam questions using Codex and then comparing these answers with the students' actual responses. Additionally, the study investigated the types of differences in Codex's responses to various problem variations and how many different types of solutions it could generate for the questions asked. The Davinci Codex model was used in the study. In the first phase of the two-stage evaluation, 23 programming tasks from an assessment exam conducted in a lab environment at the institution in 2020 were used. In the second phase, six variations of the classic "Rainfall Problem" known in the literature and one variation applied by the researchers themselves were utilized. According to the research results, Codex successfully answered all but four of the 23 questions in the first phase, where it provided correct solutions for the remaining four questions but with minor formatting errors. When compared

Table 3 Cross-tabulation of student responses in two attempts

			Second attempt		Total
			Incorrect	Correct	
First attempt	Incorrect	Count	27	3	30
		% within First Attempt	90.0%	10.0%	100.0%
		% within Second Attempt	75.0%	33.3%	66.7%
	Correct	Count	9	6	15
		% within First Attempt	60.0%	40.0%	100.0%
		% within Second Attempt	25.0%	66.7%	33.3%
Total	Count		36	9	45
	% within First Attempt		80.0%	20.0%	100.0%
	% within Second Attempt		100.0%	100.0%	100.0%

to other students, Codex ranked 17th out of 71 students, demonstrating significant performance. On the other hand, this finding is consistent with studies that show AI assistants improve student success [15, 25].

Another aspect that this research aimed to address is how students use the content generated by AI. For this purpose, the responses of students to a question where the AI provided an incorrect answer were analyzed. A majority of the students (67%) answered the question incorrectly in the first attempt, while the number of students who answered incorrectly in the second attempt increased to 80%. The findings showed that the incorrect information provided by the AI Assistant was accepted without question by the majority of the students (92%). Moreover, 61% of the students who answered incorrectly in the second attempt directly pasted the incorrect information generated by the AI (the statement that “111” is not in the range 0–255) into the answer field. When considering all the students who participated in the study, this rate is 49%, which is quite high. Students generally tend to accept the content generated by AI as correct. Especially, through the complex process of learning programming, students can easily trust the content produced by GenAI tools without a proper knowledge of learning concepts [26]. In the study conducted by Prather et al. [26], GitHub Copilot was used in a beginner (CS1) programming course to examine students’ interactions with automated code generation tools. Unlike other studies, this study investigated the effects on students who had never interacted with Copilot or any other AI tool. As a result, they found that some students endeavored to accept the code they receive from Copilot even though they find it ineffective, useless, or erroneous, so they lose time in finding the target code while they forced the tool to generate useful answers.

In this study, since students were allowed to use AI, directly taking content generated by AI was not considered plagiarism or cheating. However, the fact that a significant portion of students directly accepted the clearly erroneous statements generated by AI indicates that this could be a significant issue, especially in take-home assignments. The ability of generative AI tools to produce content in the form of human-like text, and the ability of students to share this content as if they had produced it themselves, enables cheating [27]. Our findings suggest that students transfer the critical thinking process to an AI tool by using the answers they receive from AI without questioning. In the literature, there are self-report-based studies that have reached the findings that GenAI affects students’ critical thinking by loss of creativity [28].

6 Conclusion

This paper presents the results of a preliminary investigation into the use of GenAI-based assistants in programming education. Although this study was conducted with a small group of students over a limited period of time it highlights the significance of GenAI tools integration into programming courses in Higher Education. Our findings indicate that AI assistance can increase students’ academic success. However, there are important considerations regarding their use in education. Firstly, the primary goal of a large language model is to predict the next word in a sequence. It should be considered that the content it generates is limited to the data it was trained on. Therefore, it may not always produce accurate information, the information it generates may not be up-to-date, or it may contain bias. On the other hand, plagiarism is a prevalent issue among university students, and large language models can exacerbate this problem. This is because large language models can generate unique content, and plagiarism detection tools are not yet fully capable of accurately detecting AI generated content. Erroneous content and plagiarism can lead to students learning a subject incorrectly or not learning it at all. Therefore, instead of solely researching the impact of AI on students’ academic success, it is also necessary to investigate how students use AI-generated content. It is also crucial in future studies to investigate learning designs that enable students to learn from or alongside AI. In doing so, it is necessary to develop AI tools that support students’ learning processes and test their effects.

This study also has the potential to guide researchers who wish to develop their own AI assistants. It is difficult to capture students’ AI interactions with free tools like ChatGPT. Additionally, these tools are general-purpose chatbots, and students can easily get distracted. On the other hand, custom-developed chatbots can be tailored with prompt engineering. For example, in programming education, instead of providing final code to students, they can be guided towards the target with hints. Many generative language models allow for such adjustments by using prompting techniques. Moreover, since student-AI interactions can be recorded, this can provide researchers with important insights into deeply understanding this new type of interaction.

Author contributions Both authors have contributed equally to the work.

Funding This study was not funded by any grant.

Data availability The datasets generated and analyzed during the current study are available from the corresponding author on reasonable request.

Code availability Not applicable.

Declarations

Ethics approval and consent to participate The data collection tools and methodology for this study was approved by the Human Research Ethics committee of the Hacettepe University (Ethics approval number: E-51944218-050-00003573478). Informed consent was obtained from all individual participants included in the study.

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Bennedsen J, Caspersen ME. Failure rates in introductory programming. *SIGCSE Bull.* 2007;39(2):32–6. <https://doi.org/10.1145/1272848.1272879>.
2. Bennedsen J, Caspersen ME. Failure rates in introductory programming: 12 years later. *ACM Inroads.* 2019;10(2):30–6. <https://doi.org/10.1145/3324888>.
3. Watson C, Li FWB. Failure rates in introductory programming revisited. In: *Proceedings of the 2014 conference on Innovation & technology in computer science education*, Uppsala, Sweden. 2014. <https://doi.org/10.1145/2591708.2591749>
4. Tarek M, Ashraf A, Heidar M, Eliwa E. Review of programming assignments automated assessment systems. In: *MIUCC 2022 - 2nd International Mobile, Intelligent, and Ubiquitous Computing Conference*, 2022.
5. Garcia MB. Profiling the skill mastery of introductory programming students: A cognitive diagnostic modeling approach. *Educ Inf Technol.* 2024. <https://doi.org/10.1007/s10639-024-13039-6>.
6. Humble N, Mozellius P. The threat, hype, and promise of artificial intelligence in education. *Discov Artif Intell.* 2022;2(1):22. <https://doi.org/10.1007/s44163-022-00039-z>.
7. Becker BA, Denny P, Finnie-Ansley J, Luxton-Reilly A, Prather J, Santos EA. Programming is hard - or at least it used to be: educational opportunities and challenges of AI code generation. In: *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, Toronto ON, Canada, 2023. <https://doi.org/10.1145/3545945.3569759>.
8. Jho H. Leveraging generative AI in physics education: Addressing hallucination issues in large language models [Review]. *New Phys.* 2024;74(8):812–23. <https://doi.org/10.3938/NPSM.74.812>.
9. Cotton DRE, Cotton PA, Shipway JR. Chatting and cheating: Ensuring academic integrity in the era of ChatGPT. *Innov Educ Teach Int.* 2024;61(2):228–39. <https://doi.org/10.1080/14703297.2023.2190148>.
10. Humble N, Boustedt J, Holmgren H, Milutinović G, Seipel S, Östberg A-S. Cheaters or AI-enhanced learners: Consequences of ChatGPT for programming education. *Electron J e-Learn.* 2023. <https://doi.org/10.34190/ejel.21.5.3154>.
11. Karnalim O, Toba H, Johan MC, Handoyo ED, Setiawan YD, Luwija JA. Plagiarism and AI assistance misuse in web programming: Unfair benefits and characteristics. In: *2023 IEEE international conference on teaching, assessment and learning for engineering*, Tale, 2023.
12. Tolstykh OM, Oshchepkova T. Beyond ChatGPT: Roles that artificial intelligence tools can play in an English language classroom. *Discov Artif Intell.* 2024;4(1):60. <https://doi.org/10.1007/s44163-024-00158-9>.
13. Kazemitabaar M, Chow J, Ma CKT, Ericson BJ, Weintrop D, Grossman T. Studying the effect of AI code generators on supporting novice learners in introductory programming. In: *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, Hamburg, Germany, 2023. <https://doi.org/10.1145/3544548.3580919>.
14. Okonkwo CW, Ade-Ibijola A. Python-bot: A chatbot for teaching python programming. *Eng Lett.* 2021;29(1):25–34.
15. Pankiewicz M, Baker RS. Large language models (GPT) for automating feedback on programming assignments. In: Shin AKJ-L, Chen W, Ogata H (editors). *31st International Conference on Computers in Education Conference Proceedings*. Asia-Pacific Society for Computers in Education (APSCE); 2023. Vol. I, pp. 68–77
16. Eilermann S, Wehmeier L, Niggemann O, Deuter A. KIAAA: An AI assistant for teaching programming in the field of automation. In: *2023 IEEE 21st International Conference on Industrial Informatics (INDIN)*, 18–20 July 2023.

17. Kazemitabaar M, Ye R, Wang X, Henley AZ, Denny P, Craig M, Grossman T. CodeAid: Evaluating a classroom deployment of an LLM-based programming assistant that balances student and educator needs. In: Proceedings of the CHI Conference on Human Factors in Computing Systems, Honolulu, HI, USA. 2024. <https://doi.org/10.1145/3613904.3642773>.
18. Cipriano BP, Alves P. GPT-3 vs object oriented programming assignments: An experience report. In: Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1, Turku, Finland, 2023. <https://doi.org/10.1145/3587102.3588814>.
19. Chen B, Lewis CM, West M, Zilles C. Plagiarism in the age of generative ai: cheating method change and learning loss in an intro to CS course. In: L@S 2024 - Proceedings of the 11th ACM Conference on Learning @ Scale, 2024.
20. Denzler B, Vahid F, Pang A, Salloum M. Style anomalies can suggest cheating in CS1 programs. In: Proceedings of the 2024 conference innovation and technology in computer science education, VOL 1, ITICSE 2024.
21. Taylor Z, Blair C, Glenn E, Devine TR. Plagiarism in entry-level computer science courses using ChatGPT. In: Proceedings - 2023 Congress in Computer Science, Computer Engineering, and Applied Computing, CSCE 2023, 2023.
22. Cohen J. CHAPTER 2 - The t test for means. In: Cohen J, editor. Statistical power analysis for the behavioral sciences. New York: Academic Press; 1977. p. 19–74.
23. Beukelman T, Brunner HI. Chapter 6 - Trial design, measurement, and analysis of clinical investigations. In: Petty RE, Laxer RM, Lindsley CB, Wedderburn LR, editors. Textbook of pediatric rheumatology. 7th ed. Philadelphia: W.B. Saunders; 2016. p. 54–77.
24. Finnie-Ansley J, Denny P, Becker BA, Luxton-Reilly A, Prather J. The robots are coming: exploring the implications of OpenAI codex on introductory programming. In: Proceedings of the 24th Australasian Computing Education Conference, Virtual Event, Australia, 2022. <https://doi.org/10.1145/3511861.3511863>.
25. Essel HB, Vlachopoulos D, Tachie-Menson A, Johnson EE, Baah PK. The impact of a virtual teaching assistant (chatbot) on students' learning in Ghanaian higher education. *Int J Educ Technol High Educ*. 2022;19(1):57. <https://doi.org/10.1186/s41239-022-00362-6>.
26. Prather J, Reeves BN, Denny P, Becker BA, Leinonen J, Luxton-Reilly A, Powell G, Finnie-Ansley J, Santos EA. "It's weird that it knows what i want": Usability and interactions with copilot for novice programmers. *ACM Trans Comput-Hum Interact*. 2023;31(1):1–31. <https://doi.org/10.1145/3617367>.
27. Crompton H, Burke D. The educational affordances and challenges of ChatGPT: State of the field. *TechTrends*. 2024;68(2):380–92. <https://doi.org/10.1007/s11528-024-00939-0>.
28. Smolansky A, Cram A, Radulescu C, Zeivots S, Huber E, Kizilcec RF. Educator and student perspectives on the impact of generative AI on assessments in higher education. In: Proceedings of the Tenth ACM Conference on Learning @ Scale, Copenhagen, Denmark. 2023. <https://doi.org/10.1145/3573051.3596191>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.